```
logArgument = (logMe='default') -> console.log logMe
logArgument()
# 'default'

logArgument('not the default')
# 'not the default'
```

3.4.4. Exercises

It's time for some exercises to help you explore the concepts you've just learned:

- Use `accumulate` to create a `sumFractions` function that will sum fractions supplied as strings and return a fraction as a string. For example,

  ```
  sumFractions ['2/6', '1/4']
  ```

  should return `'7/12'` or an equivalent fraction such as `'14/24'`.
- Write a `keep` function that takes an array and returns a new array containing only the element of the array that meets a condition supplied by a function. For example,

  ```
  greaterThan3 = (n) -> n > 3
  keep [1,2,3,4], greaterThan3
  ```

  should return an array containing the single value `4`.

The next important composition technique involves using functions as return values and a concept called *closures*. In order to get there, you must first understand how the scope rules work inside functions.

3.5. Scope

Remember earlier when you defined a function called `accumulate`? Suppose this function is in a program you wrote and that one day your colleague Robin works on the program and adds this doozy to it:

```
accumulate = yes
```